

Table of Contents

1. Executive Summary	2
1.1 Findings Overview	2
1.2 Audit Scope and Methodology	2
2. Critical Findings (12)	3
2.1 Backend Security	3
2.2 Financial / Compliance	4
2.3 Frontend Security	7
3. High Severity Findings (22)	7
3.1 Backend Issues	7
3.2 Frontend Issues	13
4. Medium Severity Findings (32)	14
4.1 Backend Issues	14
4.2 Frontend Issues	16
5. Positive Observations	18
6. Priority Remediation Roadmap	18
6.1 Immediate (This Week)	18
6.2 This Sprint	19
6.3 Next Sprint	19

1. Executive Summary

This report presents the findings of a comprehensive audit of the Sattabase SaaS billing platform's Stripe integration. The audit examined all raw source code across 44 backend files (~9,330 lines of Python) and 11 frontend files (~2,200 lines of Vue/TypeScript/Astro), covering controllers, services, schemas, Stripe API client modules, webhook handlers, Celery periodic tasks, currency conversion, GDPR compliance, rate limiting, error handling, and the end-user billing dashboard.

The audit scope is exclusively end-user focused: no admin dashboard features were evaluated, as those will be built separately. All findings relate to security vulnerabilities, user experience deficiencies, compliance gaps, data integrity bugs, and performance issues that directly affect the platform's paying subscribers.

1.1 Findings Overview

Severity	Count	Key Themes
CRITICAL	12	Missing auth guards, random subscription association, zero error handling in Stripe client, revenue recognition completely broken
HIGH	22	Race conditions, missing rate limits, swallowed exceptions, GDPR orphan data, stale exchange rates, broken imports
MEDIUM	32	Webhook reliability, schema validation gaps, logger bugs, dunning timer issues, currency formatting, missing PDF downloads
LOW	20	Code quality, minor UX polish, documentation, hardcoded strings, style inconsistencies

Table 1: Audit findings summary by severity

A total of **86 unique findings** were identified across the entire codebase. Of these, 12 are classified as CRITICAL and require immediate remediation to prevent data corruption, security breaches, or financial misreporting. The 22 HIGH findings represent significant risks that should be addressed within the current sprint. The MEDIUM and LOW findings represent code quality improvements and UX polish items that should be tracked in the backlog.

1.2 Audit Scope and Methodology

This audit was conducted by reading raw source code files line-by-line. No comments, docstrings, or prior audit reports were used as evidence. Each finding references specific file paths and line numbers. The audit covered the following components:

Component	Files Audited	Lines
-----------	---------------	-------

Backend Controllers	controllers.py	1,364
Backend Services	services.py	580
Backend Models	models.py	1,272
Stripe Client + Modules	13 files in stripe/	~2,700
Celery Tasks	tasks.py, celery.py	620
Error Handler + Currency	stripe_errors.py, currency_service.py	688
Frontend Components	BillingOverview.vue, PlanComparison.vue	1,444
Frontend API + Navigation	billing.ts + 7 Astro/Vue files	~700

Table 2: Audit scope breakdown

2. Critical Findings (12)

Critical findings represent immediate risks to data integrity, security, or financial compliance. These issues should be fixed before any further feature development.

2.1 Backend Security

ID	Severity	Category	Location
CTR-01	CRITICAL	Security	controllers.py:1242, 1273

Description: Two admin endpoints (get_transactions and sync_customer) have no is_staff guard. Any authenticated user can call POST /billing/admin/transactions and POST /billing/admin/sync-customer. Only refund_subscription has a staff check.

Impact: Any logged-in user can access admin-scoped financial data endpoints and trigger Stripe customer data syncs.

Recommendation: Add is_staff guard to ALL methods in BillingAdminController, or create an IsStaff permission class applied at controller level.

ID	Severity	Category	Location
CTR-02	CRITICAL	Security / Bug	controllers.py:1190-1194

Description: Admin `refund_subscription` calls `aget_subscription_for_product(request.user, product_slug)` which filters by the requesting admin's user, not the target user. The endpoint also lacks a `target_user_id` parameter.

Impact: Admins cannot refund any user's subscription except their own. The admin refund feature is completely non-functional.

Recommendation: Add a `target_user_id` or `target_email` field to refund input. Query subscription for the target user, not `request.user`.

ID	Severity	Category	Location
STP-01	CRITICAL	Reliability	stripe/client.py (all 15 functions)

Description: None of the 15 functions that call the Stripe SDK (`create_product`, `create_price`, `create_customer`, `create_checkout_session`, `retrieve_subscription`, `modify_subscription`, etc.) catch any `stripe.error.*` exceptions or network errors. Every transient Stripe outage propagates uncaught to callers.

Impact: Any transient Stripe API failure crashes the caller. In webhook handlers, events are marked failed and retried blindly. User-facing flows produce raw 500 errors instead of graceful degradation.

Recommendation: Wrap every Stripe SDK call in `try/except stripe.error.StripeError`. Distinguish `RateLimitError` (retry), `CardError` (surface to user), and `APIConnectionError` (transient). Re-raise a custom `BillingError` with a safe message.

ID	Severity	Category	Location
CH-01	CRITICAL	Bug / Security	webhooks/handlers/charge.py:32

Description: The `charge.refunded` handler does `Subscription.objects.filter(stripe_subscription_id__isnull=False).first()` which returns an arbitrary subscription from the entire database, not the one that was actually charged.

Impact: Refund records are associated with wrong subscriptions in a multi-tenant environment. Financial reports, audit trails, and customer-visible refund history are all corrupted.

Recommendation: Resolve the subscription from the charge's `payment_intent` to invoice to subscription chain instead of using `.first()`.

2.2 Financial / Compliance

ID	Severity	Category	Location
CRIT-01	CRITICAL	Configuration	celery.py beat_schedule

Description: Two of six defined Celery tasks (recognize_revenue and cleanup_stale_webhook_events) are completely absent from beat_schedule. Only 4 of 6 tasks are scheduled. These two tasks exist as decorated functions but Celery Beat will never invoke them.

Impact: Revenue recognition entries are never created automatically. Financial reporting is fundamentally broken. Webhook event logs grow unboundedly, consuming disk and degrading DB query performance.

Recommendation: Add both tasks to beat_schedule in celery.py with appropriate crontab schedules (daily 2:30 AM for revenue, weekly Sunday for cleanup).

ID	Severity	Category	Location
CRIT-02	CRITICAL	Financial Bug	tasks.py:511-517

Description: Revenue recognition uses $\text{math.ceil}(\text{price_cents} / \text{total_days})$ for non-last days, then adjusts the last day as $\text{price_cents} - (\text{daily_cents} * (\text{total_days} - 1))$. When $\text{ceil} * (\text{total_days} - 1) > \text{price_cents}$, the last day becomes negative. Example: \$1.00 plan over 31 days produces -20 cents on the last day.

Impact: Negative revenue entries corrupt financial reports. If downstream systems sum recognized revenue, totals will be wrong. Could trigger alerts or compliance issues.

Recommendation: Replace ceiling approach with proper remainder distribution: $\text{base_daily} = \text{price} // \text{days}$, $\text{remainder} = \text{price} \% \text{days}$. First remainder days get $\text{base} + 1$, rest get base. Guarantees sum = price exactly, no negative values.

ID	Severity	Category	Location
CTR-09	CRITICAL	Security	controllers.py:1176

Description: Admin refund endpoint accepts payload: dict instead of payload: RefundInputSchema. Ninja's automatic validation is completely bypassed. Manual validation at line 1202-1205 leaks Pydantic internal validation error messages to the client.

Impact: Internal schema structure exposed to users. No OpenAPI documentation for the expected request body. Potential injection of unexpected payload keys.

Recommendation: Change signature to payload: RefundInputSchema so Ninja handles validation automatically. Remove manual validation block.

ID	Severity	Category	Location
STP-02	CRITICAL	Architecture	stripe/__init__.py:53, 237

Description: The `__init__.py` directly imports `stripe` and calls `stripe.Charge.retrieve()`, bypassing `client.py` which is supposed to be the ONLY module that imports `stripe`. The invariant documented in `client.py` is violated.

Impact: If the Stripe SDK is upgraded and `Charge.retrieve` changes, this call breaks silently. Architectural guarantee of a single Stripe-coupling point is violated, making future SDK migrations riskier.

Recommendation: Add `retrieve_charge()` to `client.py`, import it into `__init__.py`, and remove the direct import `stripe` from `__init__.py`.

ID	Severity	Category	Location
IN-01	CRITICAL	Bug	webhooks/handlers/invoice.py:184, 188

Description: `handle_invoice_payment_failed` saves the subscription object twice: first setting `status=PAST_DUE` (line 184), then setting `dunning_step=0` and saving again (line 188). Between these two writes, a concurrent webhook handler could modify the same subscription, causing data loss.

Impact: Race conditions in concurrent webhook processing can cause field overwrites. The double-save pattern is unsafe.

Recommendation: Combine into a single save: set both `status` and `dunning_step` before calling `save()` once.

ID	Severity	Category	Location
IN-02	CRITICAL	Bug	webhooks/handlers/invoice.py:104-115

Description: The fee extraction code mixes `dict.get()` and attribute access on what could be either `StripeObject` or `dict`. A dead code branch checks `hasattr(fee, "to_dict")` but `fee` is always an `int` from `bt.get("fee", 0)`. The code is deeply confused about whether it works with Stripe objects or dicts.

Impact: Currently works by accident. If `stripe.BalanceTransaction.retrieve` returns a `StripeObject` and someone changes `bt.get()` to `bt.balance_transaction`, type confusion will cause a crash.

Recommendation: Route all calls through `client.py`. Add `retrieve_balance_transaction()` and `retrieve_charge()` to `client.py`.

ID	Severity	Category	Location
HIGH-03	CRITICAL	Security	settings.py:396-398

Description: All three Stripe keys (STRIPE_SECRET_KEY, STRIPE_PUBLISHABLE_KEY, STRIPE_WEBHOOK_SECRET) default to empty string. No startup validation exists. Webhook processing with empty STRIPE_WEBHOOK_SECRET means signatures are not verified.

Impact: Production deployment without env vars silently breaks all payment operations. Empty webhook secret allows forged events.

Recommendation: Add startup validation in BillingConfig.ready() that raises ImproperlyConfigured if Stripe keys are empty.

2.3 Frontend Security

ID	Severity	Category	Location
SEC-01	CRITICAL	Security	BillingOverview.vue:697, PlanComparison.vue:323, Navbar.astro:133-145, index.astro:100-104

Description: Every anchor tag with target="_blank" lacks rel="noopener noreferrer". This exposes the site to reverse tabnabbing where a malicious hosted_url from Stripe invoice data could overwrite window.opener.location.

Impact: A compromised hosted_url could redirect the user to a phishing page while the Sattabase tab stays open.

Recommendation: Add rel="noopener noreferrer" to every anchor tag with target="_blank" across all frontend files.

3. High Severity Findings (22)

High severity findings represent significant risks that should be addressed within the current sprint. These include race conditions, missing rate limits, broken error handling, and important UX deficiencies.

3.1 Backend Issues

ID	Severity	Category	Location
CTR-04	HIGH	Security / Bug	controllers.py:977-1002

Description: In `confirm_plan_change`, between fetching the fresh proration preview and verifying the preview token, the Stripe billing state could change. If amounts drift by even 1 cent, the token becomes invalid with a confusing error.

Impact: Users see confusing errors during legitimate plan changes when Stripe state changes between preview and confirm.

Recommendation: Add a tolerance window (plus or minus 1-2 cents) in token verification, or add a retry mechanism.

ID	Severity	Category	Location
CTR-05	HIGH	Security	controllers.py:808-829

Description: The `confirm_checkout` endpoint (primary subscription activation path) has no rate limiting. An attacker with a valid session ID could hammer this endpoint, consuming Stripe API quota.

Impact: Potential denial of service and increased costs from unnecessary Stripe API calls.

Recommendation: Add `check_rate_limit_or_raise(request, "confirm_checkout", max_attempts=5, window_seconds=60)`.

ID	Severity	Category	Location
CTR-06	HIGH	Security	controllers.py:401, 449, 492, 871, 841

Description: Five mutation endpoints (`cancel`, `reactivate`, `change_plan`, `preview_plan_change`, `create_portal`) have no rate limiting. Only `create_checkout` and `confirm_plan_change` are rate-limited. Each makes Stripe API calls.

Impact: An authenticated user can flood Stripe API calls through any of these endpoints, potentially triggering Stripe-side rate limits and slowing legitimate requests.

Recommendation: Add `check_rate_limit_or_raise()` to all mutation endpoints with appropriate limits.

ID	Severity	Category	Location
CTR-10	HIGH	Security	controllers.py:808

Description: `confirm_checkout` endpoint does not call `require_verified_email()`. A user who verified email, started checkout, then changed their email to an unverified one could still activate paid subscriptions.

Impact: Unverified users could bypass the email verification requirement that exists for all other mutation endpoints.

Recommendation: Add `require_verified_email(request)` at the top of `confirm_checkout`.

ID	Severity	Category	Location
SVC-01	HIGH	Bug / Race	services.py:268-297

Description: `aget_or_create_free_subscription` performs a get then create without database-level atomicity. Two concurrent requests can both hit `DoesNotExist` and both attempt to create, causing duplicate subscriptions or `IntegrityError`.

Impact: Duplicate subscriptions for the same user+product. Called on every page load with `X-Service-Domain` header, making this a high-traffic race condition.

Recommendation: Use Django's `get_or_create()` which handles the race internally, or use `select_for_update()` with `transaction.atomic()`.

ID	Severity	Category	Location
SVC-02	HIGH	Architecture	services.py:502, 553

Description: `get_auth_me_data()` calls `get_or_create_free_subscription()` which creates new Subscription records as a side effect of a semantically read-only GET request. HTTP GET should be idempotent.

Impact: Unexpected writes during GET requests. Duplicate subscription creation from browser prefetch. GDPR: creating data records on a GET is questionable from a consent perspective.

Recommendation: Remove auto-creation from `auth/me`. Auto-create subscriptions only during user registration or plan selection.

ID	Severity	Category	Location
CTR-08	HIGH	Performance	controllers.py:327-359

Description: `list_subscriptions` returns `response=list[SubscriptionOutputSchema]` with no pagination. All results are materialized. `BillingService` provides `get_subscriptions_queryset()` for pagination, but the controller ignores it.

Impact: Unbounded query results; potential performance degradation if subscription count grows across products.

Recommendation: Change to `PaginatedResponse[SubscriptionOutputSchema]` and accept `PaginationInput` query parameters.

ID	Severity	Category	Location
CL-01	HIGH	Performance	stripe/client.py:137

Description: `list_prices()` uses `auto_paging_iter()` with no upper bound, fetching ALL matching prices from Stripe. No limit parameter constrains the total iterator. Called on every checkout via `resolve_price_id`.

Impact: A product with many price variations could cause memory spikes and slow responses.

Recommendation: Break early after finding a match in `resolve_price_id`, or add a `max_results` parameter.

ID	Severity	Category	Location
CK-01	HIGH	Bug / UX	stripe/checkout.py:164-166

Description: `confirm_checkout` swallows ALL exceptions as "Invalid checkout session" via `except Exception:`. This masks real errors including `StripeError`, `ConnectionError`, and actual bugs.

Impact: If Stripe returns a 502 error, the user sees "Invalid checkout session" instead of a useful error message.

Recommendation: Catch only the expected `InvalidRequestError` and let other exceptions propagate to the caller.

ID	Severity	Category	Location
PT-01	HIGH	Security	stripe/portal.py:13, 23-29

Description: `create_portal` accepts arbitrary `return_url` with no validation. No server-side check means the system relies entirely on Stripe's client for URL validation.

Impact: If Stripe's domain configuration is permissive, an attacker could redirect users to a phishing page.

Recommendation: Validate that `return_url` hostname is in `settings.ALLOWED_HOSTS` before passing to Stripe.

ID	Severity	Category	Location
GD-01	HIGH	Compliance	stripe/gdpr.py:57

Description: GDPR anonymization clears stripe_customer_id but leaves stripe_subscription_id intact. Active Stripe subscriptions continue sending webhooks, partially restoring data that should have been erased.

Impact: GDPR Article 17 (Right to Erasure) violation. Deleted user data may be partially restored via webhooks.

Recommendation: Also clear stripe_subscription_id, or cancel the Stripe subscription before deleting the customer.

ID	Severity	Category	Location
PR-01	HIGH	Bug	stripe/prices.py:115-154

Description: Between list_prices() and create_price(), there is no locking. Two concurrent requests for the same plan+currency can both find no matching price and both create one, resulting in duplicate Stripe prices.

Impact: Duplicate prices in Stripe cause billing confusion and unnecessary API costs.

Recommendation: Use select_for_update() on the Plan row, or use Stripe's idempotency key on create_price.

ID	Severity	Category	Location
IN-03	HIGH	Bug / Compliance	webhooks/handlers/invoice.py:26-30

Description: Percentage-based discounts are explicitly skipped (commented as "skip for now"). discount_cents is always 0 for percentage-based coupons.

Impact: Financial reports show \$0 discounts for percentage-coupon invoices. Tax reconciliation is incorrect. Revenue recognition may be overstated.

Recommendation: Use invoice.get("total_discount_amounts") to get actual discount amounts per currency.

ID	Severity	Category	Location
STP-03	HIGH	Bug	stripe/__init__.py:267-269

Description: Refund idempotency key includes timestamp: f"refund-{sub.id}-{payment_intent_id}-{int(time.time())}". Since time.time() is unique per call, every retry creates a new idempotency key, defeating its purpose.

Impact: Double-refunds on network failures. The retry with a new key will attempt a second refund.

Recommendation: Use a deterministic key: f"refund-{sub.id}-{payment_intent_id}-{amount_cents}".

ID	Severity	Category	Location
STP-04	HIGH	Bug	stripe/___init___.py:557-559

Description: Plan change idempotency key also includes timestamp. Each retry creates a new key, potentially causing double-charges for plan upgrades.

Impact: If PaymentIntent succeeds but the response is lost, the retry creates a second PaymentIntent, charging twice.

Recommendation: Use deterministic key: f"plan-change-{sub.id}-{new_plan.slug}".

ID	Severity	Category	Location
HIGH-01	HIGH	Data Integrity	tasks.py:172-193, 331-334

Description: Dunning cancel step catches ALL exceptions during cancel_subscription_on_stripe(), logs the error, and returns normally. The caller then advances dunning_step to 4. Stripe subscription remains active but locally marked canceled.

Impact: Customer appears canceled locally but their Stripe subscription keeps billing silently. Manual reconciliation needed.

Recommendation: Have _execute_dunning_step return success boolean and only advance dunning_step on success.

ID	Severity	Category	Location
HIGH-02	HIGH	UX / Bug	tasks.py:154-155, 331-334

Description: Dunning step is advanced even when email fails to send. _send_dunning_email returns False on failure but the caller unconditionally sets dunning_step. The email step will never retry.

Impact: Customers may never receive payment failure emails if the first attempt fails.

Recommendation: Have _execute_dunning_step return success/failure. Only advance dunning_step when the action succeeded.

ID	Severity	Category	Location
HIGH-04	HIGH	Data Integrity	models.py:841-844, currency_service.py:308-312

Description: ExchangeRate.fetched_at uses auto_now_add=True which only sets on creation. The update_or_create in update_exchange_rates updates rate but NOT fetched_at. After first insert,

fetch_at is frozen.

Impact: Impossible to detect stale exchange rates. A rate stored 6 months ago looks identical to one stored today.

Recommendation: Include fetch_at=timezone.now() in the update_or_create defaults dict.

3.2 Frontend Issues

ID	Severity	Category	Location
SEC-02	HIGH	UX / Bug	BillingOverview.vue:753

Description: Cancel modal has a select dropdown for cancellation reason, but executeCancelModal() only passes productSlug to the API. The selected reason is completely ignored and discarded.

Impact: Users who take time to explain why they are canceling feel their input is ignored. Business loses churn data.

Recommendation: Read the select value and pass it to the backend API, or remove the dropdown to avoid deception.

ID	Severity	Category	Location
UX-01	HIGH	UX	BillingOverview.vue:239-251, 646

Description: Billing history section shows "No billing history available yet" because loadTransactions() is never called on mount. It is only invoked when the user clicks "Load More", but no initial load happens.

Impact: Users with active paid subscriptions see empty billing history and assume they have no invoices.

Recommendation: Call loadTransactions() after the initial subscription fetch in onMounted().

ID	Severity	Category	Location
UX-04	HIGH	UX	BillingOverview.vue:122-147

Description: When the initial Promise.all([getProducts(), getSubscriptions()]) fails, a toast error is shown but no error state or retry button is displayed. The user sees an empty dashboard that looks like they have no data.

Impact: After a network error, users see a confusing empty dashboard with no way to retry without page refresh.

Recommendation: Add a dedicated error state with a prominent error banner and "Retry" button.

ID	Severity	Category	Location
NAV-01	HIGH	UX / Navigation	Sidebar.astro:25, DashboardHome.vue:238

Description: Sidebar "Plans" link is hardcoded to /dashboard/billing/plans/finance. If the finance product does not exist, users see a 404. If new products are added, users cannot discover them from the sidebar.

Impact: Broken navigation if product slug changes. Users cannot discover new products.

Recommendation: Link to /dashboard/billing (overview) where all products are listed, or dynamically resolve the product slug.

4. Medium Severity Findings (32)

Medium severity findings represent important issues that should be addressed in the near term. Due to volume, this section highlights the most impactful findings. A complete list is available in the appendix.

4.1 Backend Issues

ID	Severity	Category	Location
CTR-12	MEDIUM	Bug	controllers.py:727

Description: Checkout reactivation block catches (ValueError, Exception) which swallows StripeError. If reactivation fails for a Stripe-specific reason, the user gets a generic "Contact support" message instead of actionable info.

Impact: Users get generic errors instead of specific guidance (e.g., "your card was declined").

Recommendation: Restructure to catch StripeError first and call handle_stripe_error, then catch ValueError, then Exception.

ID	Severity	Category	Location
CTR-13	MEDIUM	UX / Bug	controllers.py:466

Description: reactivate_subscription only checks status != "canceled". It does not verify that current_period_end is still in the future. A canceled subscription whose period has expired will fail with a cryptic Stripe error.

Impact: Users see confusing Stripe error instead of a clear "subscription has expired" message.

Recommendation: Check if `current_period_end < timezone.now()` and raise `BadRequestException` with a clear message.

ID	Severity	Category	Location
SVC-03	MEDIUM	Bug	services.py:349-353, 365-370

Description: Both `change_subscription_plan` and `_do_change_plan` log `old_plan={subscription.plan.slug}` AFTER calling `subscription.change_plan(new_plan)` which already reassigns plan. Audit trail shows same plan for old and new.

Impact: Audit trail is incorrect. All plan change logs show no actual change.

Recommendation: Capture `old_plan_slug` before calling `subscription.change_plan(new_plan)`.

ID	Severity	Category	Location
MED-04	MEDIUM	Bug	tasks.py:323

Description: Days past due is calculated as `(now - sub.updated_at).days`. But `updated_at` uses `auto_now=True`, so it advances each time the dunning task saves. This artificially keeps `days_past_due` low and delays step escalation.

Impact: Dunning escalation is delayed. Cancel at 14 days may never trigger.

Recommendation: Add a dedicated `past_due_at` timestamp field set once when status transitions to `PAST_DUE`.

ID	Severity	Category	Location
MED-05	MEDIUM	Compliance	tasks.py:468-474

Description: Revenue recognition query includes `PAST_DUE` status. Revenue is recognized for subscriptions where payment has failed. This violates ASC 606 revenue recognition principles.

Impact: Revenue is overstated if past-due invoices are never paid.

Recommendation: Remove `PAST_DUE` from the recognition query. Recognize retroactively when payment succeeds via webhook.

ID	Severity	Category	Location
MED-02	MEDIUM	Security	rate_limit.py:98-102

Description: get_client_ip trusts X-Forwarded-For unconditionally. An attacker can set this header to bypass rate limits by rotating spoofed IPs if not behind a properly configured proxy.

Impact: Rate limiting on billing actions (cancel, checkout, plan change) can be bypassed.

Recommendation: Use django-ipware or configure the proxy to overwrite X-Forwarded-For.

4.2 Frontend Issues

ID	Severity	Category	Location
UX-03	MEDIUM	UX	BillingOverview.vue:47-49

Description: activeSubscriptions filter includes "canceled" status, but stats.activeCount only counts "active" and "trialing". The header shows "X active" which may differ from the stat card.

Impact: Inconsistency: header shows different number than stat card for active subscriptions.

Recommendation: Align filters. Rename header label from "X active" to "X subscriptions".

ID	Severity	Category	Location
UX-13	MEDIUM	UX	billing.ts:353

Description: formatPrice uses hardcoded "en-US" locale. For BDT users, EUR users, and JPY users, number formatting is incorrect for their locale.

Impact: BDT users see dollar formatting. EUR users see comma instead of period as decimal separator. JPY shows unnecessary decimal places.

Recommendation: Detect user locale from navigator.language. For JPY and zero-decimal currencies, set minimumFractionDigits=0.

ID	Severity	Category	Location
UX-14	MEDIUM	UX	BillingOverview.vue:694-702

Description: Only hosted_url is linked for invoices. The pdf_url field exists in the schema but is never used in the template.

Impact: Users cannot download invoices locally. Standard SaaS expectation is a "Download PDF" button.

Recommendation: Add a download button using the pdf_url field next to the "View" link.

ID	Severity	Category	Location
NAV-04	MEDIUM	UX	index.astro:69, Sidebar.astro:25

Description: There is no public-facing pricing page. All plan information requires authentication. Users cannot evaluate pricing before creating an account.

Impact: Conversion friction. Potential customers cannot see plans without signing up first.

Recommendation: Create a public /pricing page using the public API endpoints.

ID	Severity	Category	Location
UX-07	MEDIUM	UX	PlanComparison.vue:202-211

Description: Downgrading to free uses a toast confirmation, while cancellation uses a full modal. A downgrade effectively IS a cancellation (loss of paid features) but gets lighter-weight confirmation.

Impact: Users may accidentally confirm downgrade via toast without understanding consequences.

Recommendation: Use a modal for paid-to-free downgrades listing features being lost.

ID	Severity	Category	Location
A11Y-01	MEDIUM	Accessibility	BillingOverview.vue:709-781, PlanComparison.vue:577-646

Description: Both modals lack focus trap, Escape key handler, role="dialog", aria-modal="true", and aria-labelledby.

Impact: Keyboard/screen reader users cannot properly interact with modals. WCAG 2.1 Level A violation.

Recommendation: Add ARIA attributes, focus trap, and Escape key handling to all modals.

ID	Severity	Category	Location
SEC-03	MEDIUM	Security	api.ts:36-38

Description: JWT access and refresh tokens are stored in localStorage. Any XSS vulnerability allows full account takeover.

Impact: If any XSS vulnerability exists, full access to user account and billing data.

Recommendation: Store tokens in httpOnly cookies. Use in-memory storage for access tokens.

5. Positive Observations

Despite the findings above, the codebase demonstrates several strong architectural decisions and security practices that are worth highlighting:

1. **Stripe-first architecture:** All subscription mutations call Stripe first, then update the local DB only on success. This prevents DB drift and is well-documented across all controller methods.
2. **PCI compliance:** No card data (PAN, CVC, expiry) is ever stored or processed locally. All payment handling uses Stripe's hosted Checkout sessions.
3. **Webhook signature verification:** Webhooks properly use `stripe.Webhook.construct_event()` with the webhook secret from settings. No unverified webhook processing.
4. **Idempotent webhook processing:** The `get_or_create` pattern with `event_id` as natural key prevents duplicate webhook processing.
5. **Safe plan change flow:** The `preview-token-confirm` flow with HMAC binding, `PaymentIntent` gating for upgrades, and rate limiting is a well-designed pattern for financial mutations.
6. **Checkout deduplication:** Cache-based double-click prevention (CMP-07) is properly implemented.
7. **Webhook reconciliation:** The `reconcile_unprocessed` Celery task with configurable age window provides a robust safety net for failed webhook processing.
8. **Audit trail:** `PlanChangeLog` with `from_plan`, `to_plan`, proration amount, `initiated_by`, and IP tracking provides a comprehensive billing audit trail.
9. **GDPR foundation:** Data export and anonymization endpoints exist with reasonable Stripe API integration. The foundation is solid but needs the gaps identified in this audit to be filled.
10. **Secret handling:** No hardcoded API keys. All Stripe keys come from environment variables via Django settings.

6. Priority Remediation Roadmap

6.1 Immediate (This Week)

Priority	Finding	Action
P0	CTR-01/03	Add <code>is_staff</code> guard to ALL admin controller methods
P0	CTR-02	Redesign admin refund to accept target user parameter

P0	HIGH-03	Add Stripe key startup validation in AppConfig.ready()
P0	SEC-01	Add rel="noopener noreferrer" to all target="_blank" links
P0	CRIT-01	Add missing Celery beat entries for revenue + cleanup tasks
P0	CRIT-02	Fix negative revenue calculation with remainder distribution
P0	CH-01	Fix random subscription association in refund webhook handler

Table 3: Immediate remediation items

6.2 This Sprint

Priority	Finding	Action
P1	STP-01	Add error handling to all client.py Stripe API calls
P1	CTR-05/06	Add rate limiting to all unrate-limited mutation endpoints
P1	CTR-09/10	Fix admin refund schema validation + email verification on confirm
P1	SVC-01	Fix race condition in aget_or_create_free_subscription
P1	STP-03/04	Fix broken idempotency keys in refund and plan change
P1	HIGH-01/02	Fix dunning step advancement logic for cancel and email actions
P1	HIGH-04	Fix ExchangeRate.fetched_at auto_now_add to auto_now
P1	IN-01/02	Fix double-save race + type confusion in webhook handlers
P1	UX-01/SEC-02	Auto-load transactions + fix cancel reason not being sent
P1	UX-04/NAV-01	Add error state with retry + fix hardcoded finance slug

Table 4: Sprint remediation items

6.3 Next Sprint

Finding	Action
---------	--------

SVC-02	Remove auto-creation from auth/me GET endpoint
GD-01	Clear stripe_subscription_id during GDPR anonymization
PT-01	Validate return_url against ALLOWED_HOSTS in create_portal
PR-01	Add locking or idempotency key to resolve_price_id
MED-04/05	Fix dunning timer + remove PAST_DUE from revenue recognition
UX-13/14	Fix locale-aware pricing + add PDF download for invoices
NAV-04	Create public pricing page
A11Y-01	Add ARIA attributes, focus trap, Escape handler to modals

Table 5: Next sprint remediation items